

SixthScreen — Complete Project Guide

Everything you need to build this project from scratch. This guide teaches you HOW to do it, not just gives you code. When you get stuck, come back here.

What Does This Project Do?

People can't decide what to watch. They scroll Netflix for 30 minutes and pick nothing.

SixthScreen solution: Ask 8 creative questions → calculate a "mood vector" → match with movies → recommend top 5.

What makes it different: Existing apps look at watch history. We look at **current mood**.

How It Works (Algorithm)

```
USER → answer 8 questions → each answer has a vector [6 numbers]
      → average of answers = MOOD VECTOR
      → each movie also has a vector (from its genres)
      → measure similarity with Cosine Similarity
      → final score = (60% match) + (40% TMDB world rating)
      → show top 5
```

Mood Vector (6 Dimensions)

```
[energy, darkness, intellect, emotion, adventure, social]
Each 0-10.
```

```
Tired + sad      → [2, 6, 5, 8, 3, 1]
Energetic        → [9, 1, 3, 4, 5, 9]
```

Cosine Similarity

```
from numpy import dot
from numpy.linalg import norm
similarity = dot(A, B) / (norm(A) * norm(B))    # -1 to 1
score = (similarity + 1) * 5                     # 0 to 10
```

Final Score

```
final = (match_score × 0.6) + (tmdb_rating × 0.4)
```

Tech Stack

Technology	Purpose	Why
Python 3.12	Backend language	NumPy for vector math
FastAPI	Web API framework	Fast, auto Swagger docs
PostgreSQL	Database	Native ARRAY type for vectors
SQLAlchemy	Python ↔ DB bridge	ORM
NumPy	Math computation	Cosine similarity
TMDB API	Movie data	1M+ movies, free
httpx	HTTP requests	Calling TMDB
Next.js	Frontend	SSR, React-based
Git	Version control	GitHub

File Structure

```
sixthscreen/                                (boşışler/ on your machine)
├── README.md                               → GitHub landing page
├── PROJECT-GUIDE.md                       → THIS FILE
├── LICENSE                                 → MIT license
├── .gitignore                             → Files to exclude from GitHub
├──
├── backend/
│   ├── .env.example                       → Example config (goes to GitHub)
│   ├── .env                               → ⚠ REAL config (NEVER push!)
│   ├── requirements.txt                   → Python packages
│   ├── venv/                             → ⚠ Virtual environment (NEVER push!)
│   ├── sql/
│   │   ├── 01_create_tables.sql → 8 tables
│   │   ├── 02_seed_questions.sql → 20 questions with vectors
│   │   └── 03_genre_mood_map.sql → 27 genre → mood mappings
│   └── app/
│       ├── main.py                       → Starts FastAPI, registers routes
│       ├── config.py                     → Reads .env settings
│       ├── database.py                   → PostgreSQL connection
│       ├── models.py                     → 8 tables as Python classes
│       ├── schemas.py                    → API input/output validation
│       ├── quiz_engine.py                → Question selection + mood calculation
│       ├── recommendation.py             → Cosine similarity + scoring
│       ├── tmdb_client.py                → TMDB API calls + caching
│       └── routes/
│           └── quiz.py                    → /api/quiz/* endpoints
```

```
├── movies.py          → /api/movies/* endpoints
├── tests/
│   ├── test_recommendation.py
│   └── test_quiz.py
├── frontend/
│   ├── app/
│   │   ├── page.js          → Home page
│   │   ├── layout.js        → Global layout
│   │   ├── globals.css      → Styles
│   │   ├── quiz/page.js     → Quiz page
│   │   └── results/page.js   → Results page
│   ├── components/
│   │   ├── QuestionCard.js  → Question card
│   │   ├── MovieCard.js     → Movie card
│   │   ├── MoodChart.js     → Mood visualization
│   │   ├── ProgressBar.js   → Progress indicator
│   │   └── Navbar.js        → Navigation bar
│   └── lib/
│       └── api.js           → Backend API calls
```

Build Steps — Do These In Order

Step 1: Install PostgreSQL

What: Database to store questions, movies, and results. **How:**

1. Download from <https://www.postgresql.org/download/windows/>
2. Run installer as Administrator (right-click → Run as administrator)
3. Set password to **postgres**, port to **5432**
4. If "database cluster initialisation failed" → set Locale to "C" or "English, United States"
5. Open pgAdmin from Start menu → connect to verify

Step 2: Get TMDB API Key

What: Free API to get movie data (posters, ratings, genres). **How:**

1. Go to <https://www.themoviedb.org/> → create account
2. Verify email → login → Settings → API → Create
3. Copy "API Key (v3 auth)"

Step 3: Create .env File

What: File that stores your secrets (never goes to GitHub). **How:**

1. Open **backend/.env.example** in your editor
2. Copy its contents
3. Create a new file called **.env** in the same **backend/** folder
4. Paste and fill in your values:

```
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/sixthscreen_db
TMDB_API_KEY=paste_your_key_here
APP_ENV=development
APP_DEBUG=true
```

Step 4: Create Database + Load Data

What: Create the database and populate it with questions and genre mappings. **How:**

1. Open pgAdmin → right-click "Databases" → Create → Database
2. Name: `sixthscreen_db` → Save
3. Right-click on `sixthscreen_db` → Query Tool
4. Open `backend/sql/01_create_tables.sql`, copy all, paste in Query Tool, click Run (▶)
5. Repeat for `02_seed_questions.sql`
6. Repeat for `03_genre_mood_map.sql`

Or via terminal (if psql is in PATH):

```
psql -U postgres -c "CREATE DATABASE sixthscreen_db;"
psql -U postgres -d sixthscreen_db -f backend/sql/01_create_tables.sql
psql -U postgres -d sixthscreen_db -f backend/sql/02_seed_questions.sql
psql -U postgres -d sixthscreen_db -f backend/sql/03_genre_mood_map.sql
```

After this step, commit to GitHub:

```
git add .
git commit -m "feat: database schema with 8 tables, 20 questions, and genre mappings"
git push
```

Step 5: Setup Python Environment

What: Install Python packages the project needs. **How:**

```
cd backend
python -m venv venv          # Create virtual environment (once)
venv\Scripts\activate       # Activate it (every time you work)
pip install -r requirements.txt # Install packages (once)
```

Step 6: Run Backend

What: Start the API server so you can test it. **How:**

```
cd backend
venv\Scripts\activate
uvicorn app.main:app --reload
```

You should see:

```
INFO:      Uvicorn running on http://127.0.0.1:8000
INFO:      Application startup complete.
```

What to do now:

1. Open <http://localhost:8000> → should show "app": "SixthScreen", "status": "running ☒"
2. Open <http://localhost:8000/docs> → Swagger UI (visual API testing tool)

After this step, commit to GitHub:

```
git add .
git commit -m "feat: backend API running with FastAPI and PostgreSQL"
git push
```

Step 7: Sync Movies from TMDb

What: Fetch ~120 popular movies/TV shows and cache them in your database. **How:**

1. Go to <http://localhost:8000/docs>
2. Find `POST /api/movies/sync`
3. Click "Try it out" → "Execute"
4. Should return: `{"message": "120 new movies/shows added", "success": true}`

Step 8: Run Your First Quiz

What: Test the full flow — questions → answers → recommendations. **How (in Swagger UI):**

1. `POST /api/quiz/start` → Execute → note the `session_id` (e.g., 1)
2. `POST /api/quiz/answer` → for each question, send:

```
{"session_id": 1, "question_id": 1, "selected_option": "b"}
```

3. `POST /api/quiz/complete` → send `{"session_id": 1}`
4. You get 5 movie recommendations with scores! 🎬

After this step, commit:

```
git add .
git commit -m "feat: TMDB integration and working quiz recommendation flow"
git push
```

Step 9: Build Frontend (YOU will write this code)

What: Create the visual interface users interact with. **How to start:**

```
cd frontend
npx -y create-next-app@latest ./ --js --no-tailwind --no-eslint --app --no-src-dir
--import-alias "@/*"
```

Then build each page. Ask me when you're ready to write:

- Home page (`app/page.js`)
- Quiz page (`app/quiz/page.js`)
- Results page (`app/results/page.js`)
- Components (QuestionCard, MovieCard, etc.)

Commit after each page you complete.

Step 10: Write Tests (YOU will write this code)

What: Verify your algorithm works correctly. **How:** Open `backend/tests/test_recommendation.py`, write pytest tests. Ask me for guidance.

Step 11: Deploy

What: Put it online so anyone can use it.

- Backend → Railway (free tier)
- Frontend → Vercel (free tier)
- Database → Railway PostgreSQL

Step 12: Final Polish

- Add screenshots to README
- Pin repo on GitHub profile
- Add topics: `python`, `fastapi`, `postgresql`, `nextjs`, `recommendation-engine`

Git & GitHub Workflow

First Time Setup

```
git init
git add .
```

```
git commit -m "feat: initial project structure"

# Create empty repo on https://github.com/new → name: sixthscreen
git remote add origin https://github.com/abb248/sixthscreen.git
git branch -M main
git push -u origin main
```

After Each Step

```
git add .
git commit -m "feat: description of what you did"
git push
```

Each push = green square on your GitHub profile.

Commit Message Format

feat:	new feature	→ "feat: add quiz page"
fix:	bug fix	→ "fix: score calculation error"
docs:	documentation	→ "docs: update README"
style:	visual change	→ "style: quiz card animations"
test:	tests	→ "test: cosine similarity tests"
refactor:	code cleanup	→ "refactor: clean quiz engine"

Security Rules

Rule	Why
Never push <code>.env</code>	Contains API keys + DB password
Never push <code>venv/</code>	200MB+ junk
Never hardcode secrets in code	Anyone can see on GitHub
Use <code>.env.example</code> for templates	Shows what's needed without revealing values
<code>git status</code> before every push	Check for leaks

API Endpoints

Method	URL	What
GET	<code>/</code>	Health check
POST	<code>/api/quiz/start</code>	Start quiz, return 8 questions
POST	<code>/api/quiz/answer</code>	Save one answer

Method	URL	What
POST	/api/quiz/complete	Finish quiz, return 5 recommendations
POST	/api/movies/sync	Fetch movies from TMDB

Database Tables (8)

Table	Stores
users	User accounts
questions	Questions + options + vectors
quiz_sessions	Each quiz attempt + mood vector
quiz_answers	Submitted answers + vectors
movies_cache	Cached movies from TMDB
recommendations	Top 5 results + scores
watch_history	User ratings + watchlist
genre_mood_map	Genre → mood vector mapping

Interview Answers

"Tell me about this project."

SixthScreen recommends movies based on subconscious mood analysis using a 6D mood vector with cosine similarity, combined with TMDB ratings for dual-layer scoring.

"Why these technologies?"

Python for NumPy vector math. PostgreSQL for native ARRAY type. FastAPI for auto Swagger docs.

"What was the hardest part?"

Balancing question vectors so no single question dominates one mood dimension.

"How would you improve it?"

- 1. AI-generated dynamic questions
- 2. Collaborative filtering from watch history
- 3. Group mode — combine multiple moods

Last updated: September 2026